

Smart Flood Early Warning System

Syed Tahmid Manzoor 1922073 Tanvir Bin Zahid 2131717 Farhan Tajwar Robe 2212496
syed.manzoar@northsouth.edu tanvir.zahid@northsouth.edu farhan.robe@northsouth.edu

Department of Electrical and Computer Engineering (ECE), School of Engineering and Physical Sciences (SEPS),
North South University, Bashundhara, Dhaka-1229, Bangladesh

Abstract

Bangladesh is hit by flood disasters almost every monsoon, but people living beside canals, drains, and low-lying areas often receive warnings only after the water has already become dangerous. In this project, we built a low-cost Smart Flood Early Warning System using the STM32F103C8T6 microcontroller so that a small community can get fast, local, and internet-free alerts. The prototype uses an HC-SR04 ultrasonic sensor to estimate water level, a DHT11 sensor to read temperature and humidity, and an analog soil moisture sensor to check ground saturation. The firmware smooths noisy sensor readings, combines them into a flood risk index, keeps a 15-day risk and soil history in flash memory, and reports the result as SAFE, WARNING, or EVACUATE. An OLED display, five LEDs, and a buzzer make the alert easy to notice, while the keypad allows simple actions such as muting the buzzer, resetting statistics, tuning sensitivity, calibrating sensors, viewing history, and waking the display from power-save mode. Since the total hardware cost is under $\text{₳}2,000$, the design is affordable enough for small flood-prone communities that need a first layer of local warning.

Keywords—embedded systems, flood detection, STM32, sensor fusion, OLED interface, flash history, real-time alert

I. INTRODUCTION

Flooding is one of the most common disasters in Bangladesh, and a large part of the country is affected during the monsoon season [1]. The problem is especially serious near local canals, drains, and low-lying neighbourhoods, where water can rise quickly before a national warning reaches residents. For these places, even a simple device placed near the water body can be useful if it gives people a few extra minutes to move valuables, warn others, or leave the area safely.

We intentionally kept the design independent of mobile networks or cloud servers because floods often come with power cuts, weak signals, and poor internet access. Instead, the prototype makes decisions locally on the STM32 and alerts nearby people through its own display, LEDs, and buzzer, following the people-centred idea used in modern early-warning guidance [6]. Large telemetry stations, such as those used by the Bangladesh Water Development Board (BWDB), are valuable but expensive [1]; this project focuses on a smaller unit with a bill of materials below $\text{₳}2,000$.

The project also brings together many parts of the CSE331L laboratory work: HAL-based peripheral setup, TIM1 microsecond timing, GPIO keypad scanning, I2C OLED communication, ADC sampling, flash memory writing, and non-blocking LED and buzzer control.

The remainder of this report is structured as follows. Section II describes the overall system architecture. Section III details the hardware and software implementation. Section IV discusses results, limitations, and future work.

II. SYSTEM OVERVIEW

The Smart Flood Early Warning System is organised into three practical blocks: an **input layer**, a **processing layer**, and an **output layer**. The STM32F103C8T6 Blue Pill connects these blocks and runs the decision logic.

Input layer. The system reads four inputs: the HC-SR04 measures the distance from the sensor to the water surface,

the DHT11 gives temperature and humidity, the soil moisture sensor reports ground saturation through ADC1, and the 4×4 keypad lets the user change pages, tune settings, and reset data.

Processing layer. The STM32 firmware is based on STM32CubeIDE HAL startup code, with our own routines added for sensor timing, filtering, risk calculation, display pages, keypad handling, and flash history. TIM1 is used as a $1 \mu\text{s}$ timer for the ultrasonic and DHT11 timing requirements. In normal operation the loop updates the sensors every 500 ms; in power-save mode it slows down to one sample every 5 s. Each reading is converted to a 0–100 score, combined into the Composite Flood Risk Index, and then passed through the alert state machine.

Output layer. The user sees the result on a 0.96-inch I2C OLED, which shows live readings, menus, calibration screens, min/max statistics, and a 15-day history page. The five LEDs give a quick visual indication of risk, and the active buzzer provides the audible warning. The buzzer is updated with non-blocking timing, so it does not stop the sensors or keypad from working.

The Composite Flood Risk Index is defined as a simple weighted score:

$$R = 0.50 W + 0.29 S + 0.15 \dot{W} + 0.03 H + 0.03 T \quad (1)$$

where W is the inverted water-level score, S is the soil-moisture score, $\dot{W}$ is the rise-rate score, H is the humidity score, and T is the temperature score. Each score is clipped to the range 0–100 using

$$\text{clip}(x) = \min(100, \max(0, x)). \quad (2)$$

The main water and rise-rate terms are normalised as

$$W = \text{clip}\left(\frac{100(D_{\text{safe}} - d)}{D_{\text{safe}} - D_{\text{evac}}}\right), \quad (3)$$

$$\dot{W} = \text{clip}\left(\frac{100v}{V_{\text{max}}}\right), \quad (4)$$

where d is the measured distance from the sensor to the water surface and v is the estimated water rise rate. The soil score comes from the calibrated soil-moisture percentage, while humidity and temperature are scaled over practical local weather ranges. The weights sum to 1.00, so R remains an easy-to-read 0–100 risk value. Water proximity and soil saturation carry most of the decision weight, while rise rate, humidity, and temperature still influence the final warning. Alert tiers are classified as follows:

- **SAFE** ($R < 40$): selected green/low-risk LED on, buzzer silent.
- **WARNING** ($40 \leq R < 75$): selected warning LED on, 2 Hz beep.
- **EVACUATE** ($R \geq 75$): selected high-risk LED blinks and buzzer toggles at 8 Hz.

Hysteresis prevents downgrade oscillation: EVACUATE is retained until $R \leq 70$, and WARNING is retained until $R \leq 35$. A least-squares trend engine also promotes the alert by one tier when the recent risk-index slope exceeds the predictive boost threshold.

Figure 1 shows the inverted water-distance score before it is combined with other sensor values. Figure 2 then shows how the final risk index changes as the water surface moves closer to the ultrasonic sensor for different soil saturation levels. Figure 3 adds the same idea as a heat map, making it easier to see why wet ground pushes the system toward WARNING or EVACUATE even before the water reaches the most dangerous distance.

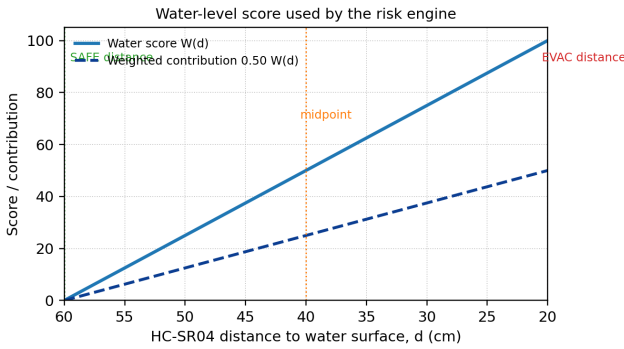


Figure 1. Normalised water score as the water surface approaches the sensor.

III. IMPLEMENTATION DETAILS

A. Hardware Implementation

The central processing unit is the STM32F103C8T6 (“Blue Pill”), an ARM Cortex-M3 microcontroller clocked at 72 MHz with 64 kB Flash and 20 kB SRAM, operating at 3.3 V logic with 5 V-tolerant GPIO pins [2,3]. The entire prototype is assembled on a 400-tie breadboard powered from a 5 V USB supply; an on-board LDO regulates the rail to 3.3 V for the MCU and peripherals.

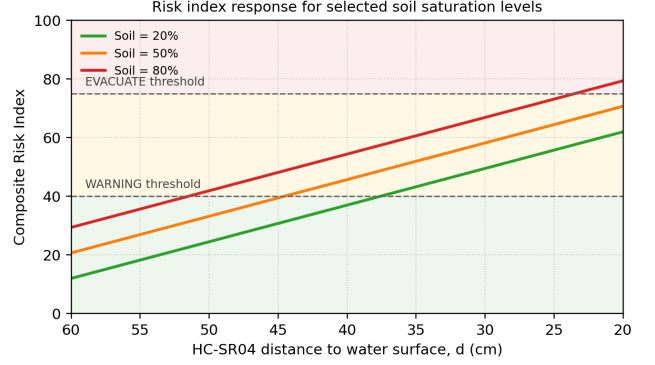


Figure 2. Composite Risk Index response for different soil saturation levels.

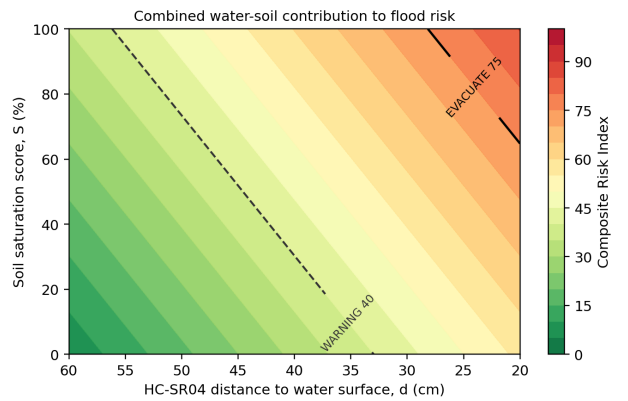


Figure 3. Risk-index heat map for water distance and soil saturation.

1) Hardware Modules and Selection Justification

Table 1 lists all hardware modules used along with selection justifications.

Table 1. Hardware Modules Used

Component	Role	Justification
STM32F103C8T6 Blue Pill	MCU	72 MHz Cortex-M3; rich peripherals (ADC, I2C, TIM, flash); low cost
HC-SR04 Ultrasonic	Water level	Non-contact; ± 1 cm; simple GPIO echo-pulse interface
DHT11	Temp/ Humidity	Single-wire; secondary atmospheric predictor; ₳120
Soil Moisture Sensor	Saturation	Analogue out; direct ADC1 connection; ₳80–120
0.96" OLED (I2C)	Display	400 kHz I2C; 3.3 V compatible; compact for field use
4×4 Keypad	User input	GPIO matrix scan; alarm silence and soft-reset
LED bank (5 LEDs)	Visual alert	Colour-coded tier; 220 Ω current limiting
Active Buzzer	Audio alert	GPIO-toggled non-blocking warning and evacuation beeps
ST-LINK V2	Programmer	SWD flash and debug interface

Table 2. Hardware Cost Breakdown (₳)

Component	Qty	Unit Price	Total
STM32 Blue Pill	1	₳410	₳410
ST-LINK V2	1	₳380	₳380
HC-SR04	1	₳110	₳110
DHT11 Module	1	₳120	₳120
Soil Moisture Sensor	1	₳100	₳100
0.96" OLED (I2C)	1	₳380	₳380
4×4 Keypad	1	₳80	₳80
Green LED	1	₳5	₳5
Yellow LED	2	₳5	₳10
Red LED	2	₳5	₳10
Active Buzzer	1	₳40	₳40
220 Ω Resistors	5	₳3	₳15
Breadboard + Jumpers	1 set	₳175	₳175
Grand Total			₳1,835

Source: Dhaka electronics market, Semester 261.

2) Hardware Cost Breakdown

Table 2 presents the complete cost breakdown.

3) Physical Layout and Pin Configuration

The prototype is built on a full-size breadboard. The Blue Pill occupies the centre rails; sensor modules are positioned along the top edge to minimise lead inductance on the HC-SR04 echo line. The LED bank and buzzer are grouped on the right side for clear visual separation. No custom PCB is required for the prototype stage. The STM32CubeMX pin map in Figure 4 keeps analogue sensing, pulse timing, I2C display traffic, alert outputs, and keypad scanning on separate GPIO groups.

Table 3 summarises the MCU pin assignments and the practical setup details used by the firmware.

B. Software Implementation

The firmware was developed in STM32CubeIDE using the STM32 HAL for peripheral setup and for most GPIO, ADC, I2C, and flash operations. For timing-sensitive parts, especially the HC-SR04 echo pulse and the DHT11 waveform, the code reads TIM1 directly. With a prescaler of 71, TIM1 increments once every 1 μ s at the 72 MHz clock speed. The main loop avoids long blocking delays: sensors are sampled every 500 ms in normal mode, the DHT11 is read every 2 s, the buzzer is toggled using elapsed-time checks, and after 40 s of stable SAFE readings the system enters power-save mode and samples every 5 s. Table 4 summarises the peripheral configuration used.

Firmware declarations. The main firmware declarations

Table 3. MCU Pin Configuration Summary

Pin(s)	Mode	Function	Details
PA0	ADC1_IN0	Soil moisture	12-bit analogue reading with 8-sample moving average
PA1	GPIO out	DHT11 data	Start pulse output, then reconfigured as input for single-wire response
PA2	GPIO out	HC-SR04 TRIG	10 μ s ultrasonic trigger pulse
PA3	GPIO in	HC-SR04 ECHO	TIM1 measures echo-pulse width with timeout checks
PA5–PA9	GPIO out	LED bank	SAFE, WARNING, and EVACUATE indicators through 220 Ω resistors
PB0	GPIO out	Active buzzer	Non-blocking alert toggle pattern for each risk tier
PB6/PB7	I2C1 SCL/SDA	OLED display	400 kHz I2C bus for the SSD1306 screen
PB8, PB10, PB11	GPIO out	Keypad drive	Row-drive outputs scanned one at a time
PB9, PB12–PB15	GPIO in	Keypad sense	Pull-up inputs for active-low key detection
PA13, PA14	SWD	ST-LINK V2	Reserved for flashing and debugging
BOOT0	Boot select	Startup mode	Held low for normal boot from flash

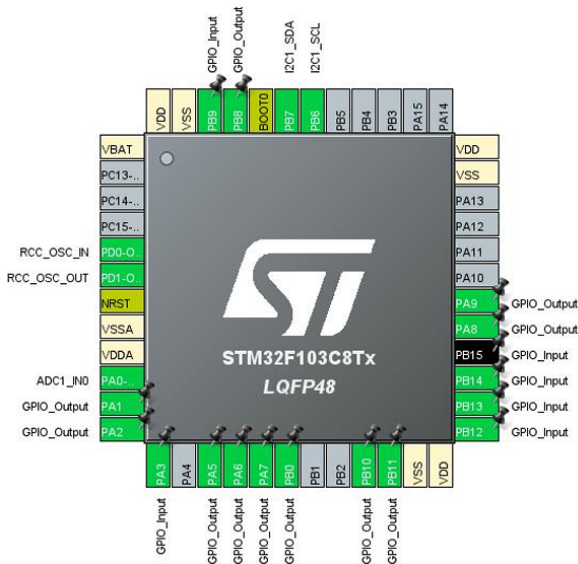


Figure 4. STM32CubeMX pin configuration for the STM32F103C8Tx LQFP48 package.

include the OLED power hooks, the shortened power-save test time, calibration constants, extra OLED screen states, soil statistics, and the 15-day flash history buffer:

```
extern void SSD1306_ON(void);
extern void SSD1306_OFF(void);

#define PSAVE_SECS 40u
#define HISTORY_DAYS 15
#define FLASH_ADDR 0x0801FC00
#define DAY_DURATION_MS 20000UL

typedef enum {
    SCREEN_MAIN,
    SCREEN_SENSORS,
    SCREEN_STATS,
    SCREEN_SENSITIVITY,
    SCREEN_ULTRA_DB,
    SCREEN_SOIL_DRY,
    SCREEN_HISTORY
} Screen_t;

typedef struct {
    float wl_min;
    float wl_max;
    float rr_min;
    float rr_max;
    uint8_t ri_min;
    uint8_t ri_max;
    uint8_t soil_min;
    uint8_t soil_max;
} Stats_t;

typedef struct {
    uint8_t avg_ri[HISTORY_DAYS];
    uint8_t avg_soil[HISTORY_DAYS];
    uint8_t count;
    uint8_t sample_loaded;
} HistoryData_t;

float water_level_display_cm = 0.0f;
float ultra_deadband_cm = 2.0f;
uint16_t soil_dry_adc = 3500;
uint8_t sys_sen = 5;
uint8_t power_save = 0;
uint32_t ultra_screen_until = 0;
uint32_t soil_screen_until = 0;
int scroll_offset = 0;
HistoryData_t history = {0};
```

The code also keeps moving-average buffers for the ultrasonic and soil readings, a 12-sample ring buffer for the recent risk-index trend, current_tier/prev_tier variables for hysteresis, and timeout values for temporary calibration screens. The keypad tuning limits are kept small and safe: ULTRA_DB_MIN=0.50 cm, ULTRA_DB_MAX=3.00 cm, SOIL_DRY_MIN=500, and SOIL_DRY_MAX=4095.

Sensor filtering, calibration, statistics, and power save. The ultrasonic reading is deliberately filtered because single HC-SR04 readings can jump when the water surface is uneven. The code sends a 10 μ s trigger pulse, records five echo measurements with TIM1, sorts them, and uses the middle value. That median value is then passed through an eight-sample moving average. Soil moisture uses a similar eight-sample average on ADC1. The wet point is fixed at SOIL_WET_ADC, while the dry point can be adjusted from the keypad using soil_dry_adc. For the display, distance is inverted against the 9 cm demonstration tank depth so the OLED shows a more intuitive water-level value. The statistics page tracks water, rise-rate, risk-index, and soil min/max values, and the OLED is turned off or back on when entering or leaving power-save mode:

```
uint8_t Soil_Read(void) {
    uint16_t raw = ADC1_Read(CH_SOIL);
    uint16_t avg = MovAvg8_Push(&soil_avg, raw);
    uint16_t dry = soil_dry_adc;

    if (dry <= SOIL_WET_ADC) dry = SOIL_WET_ADC + 1;
    if (avg > dry) avg = dry;
    if (avg < SOIL_WET_ADC) avg = SOIL_WET_ADC;

    return (uint8_t)(100 - ((avg - SOIL_WET_ADC)
        * 100 / (dry - SOIL_WET_ADC)));
}

float WaterLevel_Display(float distance_cm) {
    float wl = WL_DEPTH_CM - distance_cm;
    if (wl < 0.0f) wl = 0.0f;
    if (wl > WL_DEPTH_CM) wl = WL_DEPTH_CM;
    return wl;
}

void Stats_Reset(void) {
    stats.wl_min = 9999.0f;
    stats.wl_max = 0.0f;
    stats.rr_min = 9999.0f;
    stats.rr_max = 0.0f;
    stats.ri_min = 255;
    stats.ri_max = 0;
    stats.soil_min = 255;
    stats.soil_max = 0;
}

void Stats_Update(float wl, float rr, uint8_t ri,
    uint8_t soil) {
    if (wl < stats.wl_min) stats.wl_min = wl;
    if (wl > stats.wl_max) stats.wl_max = wl;
    if (rr < stats.rr_min) stats.rr_min = rr;
    if (rr > stats.rr_max) stats.rr_max = rr;
    if (ri < stats.ri_min) stats.ri_min = ri;
    if (ri > stats.ri_max) stats.ri_max = ri;
    if (soil < stats.soil_min) stats.soil_min = soil;
    if (soil > stats.soil_max) stats.soil_max = soil;
}

void EnterPowerSave(void) {
    power_save = 1;
    SSD1306_OFF();
    LED_AllOff();
    Buzzer_Off();
}

void ExitPowerSave(void) {
    power_save = 0;
    SSD1306_ON();
    OLED_Refresh();
}
```

LED tier handling. The LED driver first clears all outputs, exits early in power-save mode, selects exactly one active LED from the current risk index, and blinks only that LED during EVACUATE. This removes the previous multiplexing dependency on count, avoiding any divide-by-zero path.

```
void LED_SetTier(AlertTier_t tier) {
    uint8_t active_led;

    LED_AllOff();
    if (power_save) return;

    if (risk_index < 20) active_led = LED_GREEN;
    else if (risk_index < 40) active_led = LED_YELLOW1;
    else if (risk_index < 60) active_led = LED_YELLOW2;
    else if (risk_index < 80) active_led = LED_RED1;
```

```

else
    active_led = LED_RED2;

if (tier == EVACUATE) {
    if ((HAL_GetTick() / 250u) % 2u) LED_On(active_led);
} else {
    LED_On(active_led);
}
}

```

History storage. To make the history feature easy to test during a short demonstration, one “day” is treated as 20 s. Every simulated day, the firmware shifts the 15-entry arrays and saves the latest risk index and soil percentage at flash address 0x0801FC00. On the first run, the code loads representative sample values so the history page is not empty during a demo. If the user presses # while viewing history, the arrays are cleared and the seed flag is kept set so the sample data do not return immediately. The save routine is written to use only the rounded-up structure length in words, avoiding an extra write past HistoryData_t.

```

static uint8_t Avg_U8(uint8_t *arr, uint8_t count) {
    uint32_t sum = 0;
    if (count == 0) return 0;
    for (uint8_t i = 0; i < count; i++) sum += arr[i];
    return (uint8_t)(sum / count);
}

static uint8_t History_Valid(const HistoryData_t *h) {
    if (h->count > HISTORY_DAYS) return 0;
    if (h->sample_loaded > 1) return 0;
    for (uint8_t i = 0; i < h->count; i++) {
        if (h->avg_ri[i] > 100 || h->avg_soil[i] > 100) return 0;
    }
    return 1;
}

void Load_History_From_Flash(void) {
    const HistoryData_t *stored =
        (const HistoryData_t *)FLASH_ADDR;
    if (stored->count == 0xFF || !History_Valid(stored)) {
        memset(&history, 0, sizeof(history));
        return;
    }
    memcpy(&history, stored, sizeof(history));
}

void Save_History_To_Flash(void) {
    const uint32_t *src = (const uint32_t *)&history;
    uint32_t words = (sizeof(history) + 3u) / 4u;

    FLASH_Unlock();
    FLASH_ErasePage(FLASH_ADDR);
    for (uint32_t i = 0; i < words; i++) {
        FLASH_ProgramWord(FLASH_ADDR + i * 4u, src[i]);
    }
    FLASH_Lock();
}

```

OLED screens. Since the SSD1306 display is monochrome [7], the firmware uses inversion and flashing during EVACUATE to make the screen look more like a high-risk warning. The main page shows the alert tier, display water level, rise rate, DHT11 status, soil percentage, risk index, a patterned risk bar, and either the sensitivity hint or power-save message. Separate pages show live sensor data, min/max statistics, sensitivity, ultrasonic deadband, soil dry calibration, and the 15-day history table.

```

void OLED_MainScreen(void) {
    OLED_Print(0, 0, TierText(current_tier));
    OLED_PrintFloat(0, 12, "WL:", water_level_display_cm);
    OLED_PrintFloat(64, 12, "RR:", rise_rate_cms);
    OLED_PrintDHTorError(0, 24, dht_ok,
        temperature_c, humidity_pct);
    OLED_PrintU8(0, 36, "SOIL:", soil_pct);
    OLED_PrintU8(64, 36, "RI:", risk_index);
    OLED_DrawRiskBar(0, 47, 126, 5, risk_index);
    OLED_Print(0, 54, power_save ? "PWR-SAVE [*]=wake"
        : "SYS_SEN: 5/6");
}

void OLED_StatsScreen(void) {
    OLED_Print(0, 0, "MIN/MAX KEY 4");
    OLED_PrintFloat(0, 12, "WLmin:", stats.wl_min);
    OLED_PrintFloat(64, 12, "WLmax:", stats.wl_max);
}

```

```

OLED_PrintFloat(0, 24, "RRmin:", stats.rr_min);
OLED_PrintFloat(64, 24, "RRmax:", stats.rr_max);
OLED_PrintU8(0, 36, "RImin:", stats.ri_min);
OLED_PrintU8(64, 36, "RImax:", stats.ri_max);
OLED_PrintU8(0, 48, "SoilMin:", stats.soil_min);
OLED_PrintU8(64, 48, "SoilMax:", stats.soil_max);
}

void OLED_SensorScreen(void) {
    OLED_PrintFloat(0, 0, "RawWL:", water_level_cm);
    OLED_PrintFloat(0, 12, "ShowWL:", water_level_display_cm);
    OLED_PrintU8(0, 24, "Hum:", humidity_pct);
    OLED_PrintU8(0, 36, "Soil:", soil_pct);
    OLED_Print(0, 52, "A/7 Ultra 0/8 Soil");
}

void OLED_SensitivityScreen(void) {
    OLED_Print(0, 0, "SENSITIVITY");
    OLED_PrintU8(0, 16, "sys_sen:", sys_sen);
    OLED_Print(0, 48, "A/7 Ultra 0/8 Soil");
}

void OLED_UltraDeadbandScreen(void) {
    OLED_Print(0, 0, "ULTRA DEADBAND");
    OLED_PrintFloat(0, 20, "cm:", ultra_deadband_cm);
    OLED_Print(0, 52, "7:- A:");
}

void OLED_SoilDryScreen(void) {
    OLED_Print(0, 0, "SOIL DRY CAL");
    OLED_PrintU16(0, 20, "ADC:", soil_dry_adc);
    OLED_Print(0, 52, "8:- 0:");
}

void OLED_HistoryScreen(void) {
    OLED_Print(0, 0, "15-DAY HISTORY [9]");
    OLED_Print(0, 12, "Day RI Soil");
    for (uint8_t row = 0; row < 3; row++) {
        uint8_t idx = scroll_offset + row;
        if (idx >= history.count) break;
        OLED_PrintHistoryRow(row, idx + 1,
            history.avg_ri[idx], history.avg_soil[idx]);
    }
    OLED_PrintAvg(0, 46, Avg_U8(history.avg_ri, history.count),
        Avg_U8(history.avg_soil, history.count));
    OLED_Print(0, 56, "C/D:Scroll #:Reset");
}

```

Timing model. The firmware uses two time scales. The HAL millisecond tick schedules normal tasks such as sensor sampling, display refresh, buzzer updates, and power-save counting. TIM1 handles the short microsecond delays needed by the ultrasonic sensor and DHT11. The helper below resets TIM1 and waits until the required number of microseconds has passed:

```

static void micro_delay(uint16_t us) {
    __HAL_TIM_SET_COUNTER(&htim1, 0);
    while (__HAL_TIM_GET_COUNTER(&htim1) < us);
}

```

Main loop. After startup, the loop loads the saved history and then runs the same sequence repeatedly: update the simulated day log when needed, sample sensors at the current interval, calculate risk and trend, update power-save state, refresh the selected OLED page, and finally check the keypad:

```

Load_History_From_Flash();
uint32_t day_timer = HAL_GetTick();

while (1) {
    uint32_t now = HAL_GetTick();
    uint32_t sample_interval = power_save ? PSAVE_MS :
        SAMPLE_MS;

    if (now - day_timer >= DAY_DURATION_MS) {
        day_timer = now;
        for (uint8_t i = 0; i < HISTORY_DAYS - 1; i++) {
            history.avg_ri[i] = history.avg_ri[i + 1];
            history.avg_soil[i] = history.avg_soil[i + 1];
        }
        history.avg_ri[HISTORY_DAYS - 1] = risk_index;
        history.avg_soil[HISTORY_DAYS - 1] = soil_pct;
        if (history.count < HISTORY_DAYS) history.count++;
        Save_History_To_Flash();
    }

    if (now - last_sample_ms >= sample_interval) {
        last_sample_ms = now;
        float prev_wl = water_level_cm;

        float raw_wl = HCSR04_Read_cm();
        if (raw_wl < ULTRA_MIN_CM || raw_wl > ULTRA_MAX_CM)

```



```

    raw_wl = water_level_cm;
    raw_wl = WaterAvg_Push(raw_wl);

    if (!ultrasonic_ready) {
        water_level_cm = raw_wl;
        ultrasonic_ready = 1;
    } else {
        float alpha = 0.20f + 0.08f * (float)sys_sen;
        if (alpha > 1.0f) alpha = 1.0f;
        if (fabsf(raw_wl - water_level_cm) <
            ultra_deadband_cm)
            raw_wl = water_level_cm;
        water_level_cm += alpha * (raw_wl - water_level_cm)
    }

    if (now - last_dht_ms >= DHT_SAMPLE_MS) {
        last_dht_ms = now;
        dht_ok = DHT11_Read(&temperature_c, &humidity_pct);
    }
    soil_pct = Soil_Read();
    water_level_display_cm = WaterLevel_Display(
        water_level_cm);

    float dt_s = (float)sample_interval / 1000.0f;
    rise_rate_cms = (prev_wl > water_level_cm)
        ? (prev_wl - water_level_cm) / dt_s : 0.0
    f;

    uint8_t wl_s = Normalise(WL_SAFE_CM - water_level_cm,
        0.0f, WL_SAFE_CM - WL_EVAC_CM
    );
    uint8_t rr_s = Normalise(rise_rate_cms, 0.0f,
        MAX_RISE_CMS);
    uint8_t hum_s = Normalise(humidity_pct, 60.0f, 95.0f);
    uint8_t tmp_s = Normalise(temperature_c, 25.0f, 35.0f);

    risk_index = (uint8_t)(0.50f * wl_s + 0.29f * soil_pct
        + 0.15f * rr_s + 0.03f * hum_s
        + 0.03f * tmp_s);
    if (risk_index > 100) risk_index = 100;

    RingBuf_Push(&ri_buf, risk_index);
    float trend = LeastSquaresSlope(&ri_buf);
    prev_tier = current_tier;
    current_tier = Classify(risk_index, trend);
    if (current_tier > prev_tier) buzzer_muted = 0;
    Stats_Update(water_level_display_cm, rise_rate_cms,
        risk_index, soil_pct);
}

if (now - last_sec_ms >= 1000u) {
    last_sec_ms = now;
    if (current_tier == TIER_SAFE && ++stable_secs >=
        PSAVE_SECS)
        EnterPowerSave();
    else if (current_tier != TIER_SAFE)
        ExitPowerSave();
}

LED_SetTier(current_tier);
Buzzer_Update(current_tier);
OLED_DrawCurrentScreen();

switch (Keypad_Scan()) {
    case '1': buzzer_muted = !buzzer_muted; break;
    case '2':
        if (current_screen == SCREEN_STATS) Stats_Reset();
        else NVIC_SystemReset();
        break;
    case '3': ToggleMainSensorScreen(); break;
    case '4': current_screen = SCREEN_STATS; break;
    case '5': AdjustSensitivity(-1); break;
    case '6': AdjustSensitivity(+1); break;
    case '7': AdjustUltraDeadband(-1); break;
    case 'A': AdjustUltraDeadband(+1); break;
    case '8': AdjustSoilDry(-1); break;
    case '0': AdjustSoilDry(+1); break;
    case '9': ToggleHistoryScreen(); break;
    case 'C': HistoryScrollUp(); break;
    case 'D': HistoryScrollDown(); break;
    case '#': HistoryResetIfOpen(); break;
    case '*': ExitPowerSave(); break;
}
}

```

The keypad mapping used by the firmware is:

- **1**: toggle buzzer mute; **2**: reset statistics on the stats page or reset the MCU elsewhere.
- **3**: toggle main/sensor pages; **4**: open statistics; **5/6**: decrease/increase sys_sen.
- **7/A**: decrease/increase ultrasonic deadband; **8/0**: decrease/increase soil dry ADC calibration.
- **9**: open/close history; **C/D**: scroll history; **#**: clear history;

*: wake from power-save.

Alert thresholds (WL_SAFE, WL_EVAC, MAX_RISE) are defined as compile-time constants, allowing the firmware to be redeployed at a different site by adjusting only these values.

Peripheral configuration. Table 4 summarises the peripheral settings used by the HAL initialisation code and the main loop.

Table 4. Peripheral Configuration

Peripheral	Purpose	Configuration
HAL tick	Scheduler timing	1 ms HAL_GetTick () time base
TIM1	Microsecond timing	Prescaler 71, 1 μ s counter tick
GPIO PA2/PA3	HC-SR04	Trigger output / echo input with timeout checks
GPIO PA1	DHT11 data	Runtime output/input reconfiguration
ADC1 CH0	Soil moisture	12-bit ADC with 8-sample moving average
I2C1	OLED display	400 kHz fast mode, PB6/PB7
Flash page	History storage	0x0801FC00, 15-day RI/soil log
GPIO PB0	Buzzer	Non-blocking toggle at tier-specific intervals
GPIO PB8–PB15	Keypad matrix	PB8/PB10/PB11 row outputs; PB9/PB12–PB15 pull-up inputs
GPIO PA5–PA9	LED bank (5)	Push-pull alert indicators

IV. DISCUSSION

Expected outcomes. In testing, the main goal is for the HC-SR04 to track water movement smoothly after the median and moving-average filters are applied. The risk index should rise when the water comes closer to the sensor, when soil moisture increases, or when the rise rate becomes large. Hysteresis and the trend check are included so the alert does not jump back and forth near a threshold. The OLED, LEDs, and buzzer should give a clear warning without any internet connection, and the history, statistics, and calibration pages should make the prototype easier to demonstrate and tune.

Extensibility. The same firmware can be reused at another location by changing the threshold constants and then tuning sensitivity, ultrasonic deadband, and soil dry calibration from the keypad. This keeps setup simple for an identical hardware unit. Because the parts cost stays below £2,000, the prototype could be copied for several small flood-prone areas instead of being limited to a single expensive monitoring point.

Limitations. The HC-SR04 ultrasonic sensor is prone to errors in real flood conditions. Turbulent water, foam, floating debris, rain splash, or a poor mounting angle can weaken or scatter the echo, so filtering reduces noise but cannot remove every false reading. The DHT11 is also a low-cost sensor, so its humidity and temperature readings are not very precise.

The soil-moisture probe needs periodic calibration and may drift or corrode if it stays in wet soil for a long time.

Another practical limitation is the limited flash memory of the STM32F103C8T6. The current 20-second history interval is useful for demonstration, but frequent logging can consume storage and increase flash wear; a field version should log less often or use external memory. The prototype also does not yet include a weatherproof enclosure, battery backup, or long-range communication, so it must be installed where it is protected from rain and where nearby people can see or hear the alert. In future work, the DHT11 could be replaced with a DHT22 [8], the water-level sensor could be made waterproof or pressure-based, external EEPROM/FRAM or an SD card could store history, and a LoRa module could provide long-range alerts without requiring internet service [9].

ACKNOWLEDGMENT

The authors would like to thank their faculty supervisor Acram Kabir (ACQ) and lab instructor Moshir Rahman for their guidance throughout the CSE331L laboratory sessions at North South University.

A. OBJECTIVES

The following specific objectives guided the system design:

1. Continuously measure water level using an HC-SR04 ultrasonic sensor with ± 1 cm resolution via GPIO echo-pulse timing.
2. Compute real-time water rise rate (cm/s) using HAL tick scheduling and TIM1-assisted ultrasonic timing to distinguish slow seepage from rapid flash-flood surges.
3. Monitor ambient humidity and temperature via DHT11 as a secondary atmospheric flood predictor.
4. Measure soil saturation via an analogue soil moisture sensor through the STM32 ADC to assess ground absorption capacity.
5. Fuse all sensor data into a Composite Flood Risk Index and classify conditions into three tiers: SAFE, WARNING, and EVACUATE.
6. Drive a multi-modal output system comprising an OLED live display, tiered LED bank, and a buzzer with distinct tone patterns per alert tier.
7. Allow field-configurable alarm silencing and system reset via a 4×4 keypad.

REFERENCES

- [1] Bangladesh Water Development Board, “*Annual Flood Report 2022*,” Ministry of Water Resources, Government of Bangladesh, 2022. [Online]. Available: <https://www.bwdb.gov.bd>
- [2] STMicroelectronics, “*STM32F103x8/xB Datasheet*,” DS5319, Rev. 17, 2021. [Online]. Available: <https://www.st.com>
- [3] STMicroelectronics, “*RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx Advanced Arm-Based 32-Bit MCUs*,” 2024. [Online]. Available: https://www.st.com/resource/en/reference_manual/
- [4] Elec Freaks, “*HC-SR04 Ultrasonic Sensor User Guide*,” 2013. [Online]. Available: <https://www.electfreaks.com>
- [5] Aosong Electronics, “*DHT11 Humidity and Temperature Sensor Datasheet*,” 2010. [Online]. Available: <https://www.aosong.com>
- [6] World Meteorological Organization, “*Early Warnings for All*,” 2024. [Online]. Available: <https://wmo.int/activities/early-warnings-all>
- [7] Solomon Systech, “*SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*,” Rev. 1.1, 2008. [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
- [8] Aosong Electronics, “*AM2302/DHT22 Digital Temperature and Humidity Sensor Datasheet*,” 2015. [Online]. Available: <https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf>
- [9] Semtech Corporation, “*SX1276/77/78/79—137 MHz to 1020 MHz Low Power Long Range Transceiver Datasheet*,” Rev. 7, 2020. [Online]. Available: <https://www.semtech.com/products/wireless-rf/lora-transceivers/sx1276>